ASSIGNMENT 1

TECHNOLOGY PARK MALAYSIA
CT051-3-3-SQE
SOFTWARE QUALITY ENGINEERING

Software Quality Engineering Evaluation of Actual Budget: Methodology, Quality Assurance and IV&V

HAND OUT DATE: 11-09-2026

HAND IN DATE: 30-10-2026

WEIGHTAGE: 40%

INSTRUCTIONS TO CANDIDATES:

1. **Submit your assignment online using Moodle submission link**
2. **Students are advised to underpin their answers with the use of references (cited using the APA 7 format of Referencing)**
3. **Late submission will be awarded zero (0) unless Extenuating Circumstances (EC) are upheld**
4. **Cases of plagiarism will be penalized**
5. **Where the assignment should be submitted in softcopy, the softcopy of the written assignment and source code (where appropriate).**
6. **You must obtain 50% overall to pass this module.**



Module Code	**:**	**CT051-3-3-SQE**
Intake Code	**:**	**APU3F2609SE**
Lecturer Name	**:**	**Dr. Kesava Pillai A/L Rajadorai @ Rajoo**
Hand-In-Date	**:**	**30 October 2026**
Hand-Out-Date	**:**	**11 September 2026**
Student Name	**:**	**Tee Xing Yang**
Student ID	**:**	**TP076174**

Table of Contents

Executive Summary **1**
1.0 Introduction **2**
1.1 Background and Purpose 2
1.2 What Is Actual Budget 2
1.3 Main Features and How It Works 2
1.4 Who Is Involved 3
1.5 Main Challenges 3
2.0 Software Development Methodology **4**
2.1 Current Development Process of Actual 4
2.2 Why Kanban Fits Actual 5
2.3 Applying Kanban to Actual 6
2.4 Critical Evaluation 7
3.0 Quality Assurance Process **9**
3.1 What Quality Means for Actual 9
3.2 Key Quality Attributes 9
3.3 Current QA Practices and Gaps 10
3.4 Selected Standards and Practices 11
3.5 Applying the QA Process within Kanban 11
3.6 Critical Evaluation 13
4.0 Integrating IV&V with Kanban **14**
4.1 Current V&V in Actual and Its Independence 14
4.2 IV&V Integration Model for Kanban 15
4.3 How the Integration Meets the Three Goals 16
References **18**
Appendix A: Methodology Selection **21**
A.1 Project Characteristics and Requirements 21
A.2 Candidate Methodologies 22
A.3 Evaluation and Selection 22
Appendix B: SQAP Summary Poster **25**

Executive Summary

This report evaluates Actual Budget, a free, open-source and local-first personal finance application run by a small volunteer team, whose users must be able to trust its numbers and the security of their data.

Actual has a defined workflow but no deliberate methodology. Its profile suits an agile approach, except that its few reviewers are overloaded and its defects can affect money and data. Out of five candidates, we chose Kanban because it limits the review backlog, and then mandatory quality gates are added because Kanban prescribes no quality practices.

Furthermore, The QA process focuses on three ISO/IEC 25010:2023 characteristics: Security, Reliability and Interaction capability. Public evidence shows no written QA plan and gaps in proactive security verification, coverage measurement and accessibility testing. The plan also adopts OWASP ASVS 5.0.0 and WCAG 2.2 and writes risk-scaled checks into the exit policy of each Kanban column.

Lastly, IV&V adds risk-tiered independent review inside the team, plus external checks through coordinated disclosure and an optional funded audit. The outside checks are kept because all three 2026 security flaws were found by reporters outside the team, after release.

1.0 Introduction

1.1 Background and Purpose

Personal finance applications help people record spending and plan a budget. Because they hold balances, transactions and sometimes bank connections, their quality matters. A wrong total can lead to a poor financial decision, while a security flaw can expose private data.

I chose Actual Budget because I was looking for an open-source personal finance application that I could self-host on an old laptop at home to manage and keep track of my finances. And this assignment allowed me to examine it in depth.

This report covers three topics. Section 2 selects a software development methodology for the project, Section 3 sets out a quality assurance (QA) process, and Section 4 integrates independent verification and validation (IV&V) into the selected methodology.

1.2 What Is Actual Budget

Actual Budget ("Actual") is a free, open-source personal finance application released under the MIT licence. Its focus is mainly on local-first, and the budget is kept on the user's own device, while syncing between devices is optional (Actual Budget, 2026a). It targets privacy-minded budgeters, self-hosters and families. Furthermore, the project defines success as users trusting its numbers (Actual Budget, 2026a). By default, Actual uses envelope budgeting, in which people budget the money they already have, rather than the income they expect to receive (Actual Budget, n.d.-j).

Actual was created by James Long, who ran it as a paid product for over four years (Stacoviak & Santo, 2022). In early 2022 he decided to close the commercial product and released it as open source, and it is now maintained by volunteers, which led by a core team that formed after pull requests had waited months for review (Aboltins, 2025).

1.3 Main Features and How It Works

Actual covers budgeting, accounts and transactions, rules, scheduled transactions and reports (Actual Budget, n.d.-j). It runs as a desktop app, through Docker or on paid hosting from PikaPods or Fly.io (Actual Budget, n.d.-c).

Basic budgeting does not require a server, but syncing between devices and bank synchronisation both will need one (Actual Budget, n.d.-c). Users can enable end-to-end encryption for synced data, but it does not protect data stored on the local device or the bank-sync tokens held on the server (Actual Budget, n.d.-h). Figure 1 shows how the parts connect.

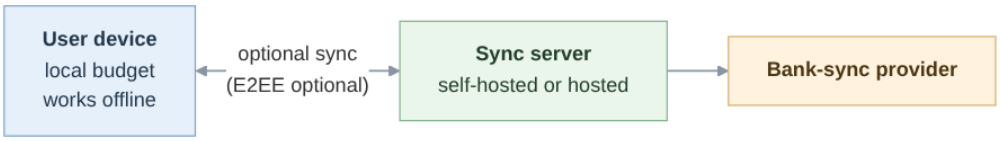


Figure 1. How Actual Budget works.

1.4 Who Is Involved

Stakeholder	Role and interest
Users	Need correct figures and private data
Self-hosters	Run their own server and need it secure
Core maintainers	Eight listed maintainers who review and release (Actual Budget, n.d.-a)
Contributors	Volunteers who choose their own work (Actual Budget, n.d.-a)
Donors and hosting partners	Fund the project through Open Collective (Actual Budget, n.d.-e); PikaPods reported over 8,000 users in December 2025 (youngcw, 2025)

Table 1.1. Main stakeholders of Actual Budget.

1.5 Main Challenges

1. **Volunteer capacity and changing requirements.**
   Work depends on volunteers, and reviews have been a bottleneck before (Aboltins, 2025).
2. **Trust in the numbers and data security.**
   Users rely on correct figures and protected data.
3. **Escaped defects and security flaws.**
   Problems found after releasing harm users.

2.0 Software Development Methodology

Because a methodology can only be judged against the project it must serve, this section first describes how Actual works today. It then explains why Kanban, tailored with quality gates, fits Actual better than Spiral, Scrum, Extreme Programming (XP) or DevOps, and then shows how it would work in practice and examines its weaknesses.

2.1 Current Development Process of Actual

Actual's contributor documentation does not name any methodology (no evidence found, October 2026). However, it does describe the same workflow for every change, shown in Figure 2.

Users' feature requests are ranked by votes, and nobody is assigned to those requests by force. Contributors can choose which to work on and submit a pull request (PR). Continuous integration (CI) and a core maintainer will then check the PR, and a monthly release train ships whatever is ready (Actual Budget, 2026a, n.d.-a, n.d.-b, n.d.-d).

The step that has caused trouble is review. Because only maintainers can approve a PR, and the project lists eight of them as of October 2026 (Actual Budget, n.d.-a). After Actual became open source in 2022, there were PRs that have been waited months because nobody was available to review them (Aboltins, 2025). The project now starts to pay maintainers for review and triage from a 2,000 USD monthly pool funded by donations (Actual Budget, n.d.-e). This shows that the review time is its scarcest resource.

Therefore, Actual has a defined workflow, but no deliberate methodology. The steps are clear, but nothing manages how work moves through them, and nothing limits how many PRs can pile up waiting for the few people that are able to review them.

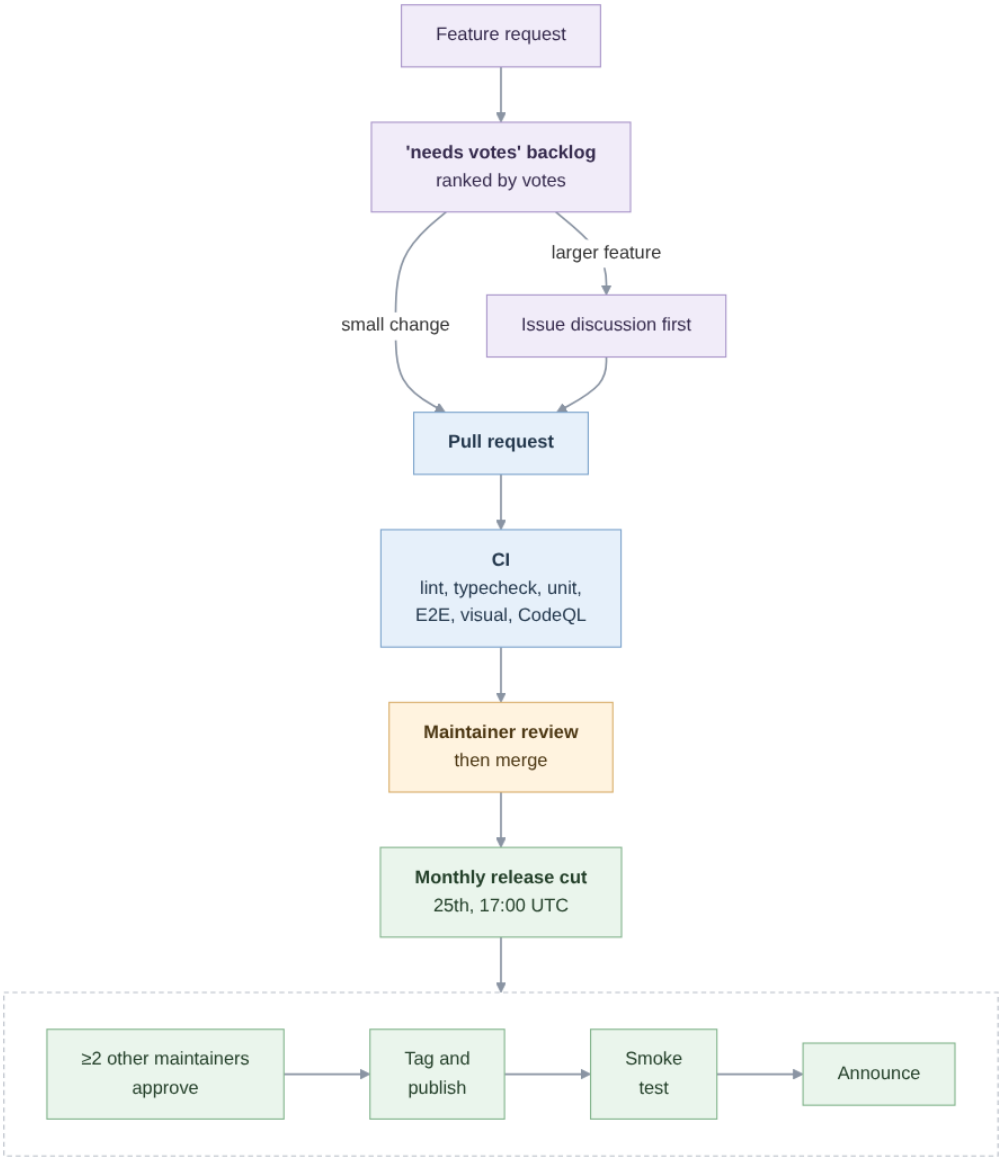


Figure 2. Actual's as-is development workflow.

2.2 Why Kanban Fits Actual

A software process model is commonly chosen by weighing eight factors about the project, such as project requirements, risk tolerance, team expertise and more. For Actual, most of these factors point to an agile model rather than a linear one such as Waterfall, as the project's requirements change continuously as users vote. Users are also closely involved because their votes shape the work, and each monthly release ships whatever is ready rather than a fixed plan. However, there are two factors that weaken the case, because agile methods rely on an experienced team and low risk tolerance favours linear models. On team expertise, the few experienced reviewers are volunteers whose time has already run out once (Aboltins, 2025). On risk tolerance, Actual cannot accept much risk, because a wrong balance could lead a household to a poor spending decision, and a security flaw can expose private financial data.

There was already a case where a critical flaw that allow anyone read an users' bank balances and transactions without the need to log in at all (Actual Budget, 2026c).

From this profile, the methodology must meet five needs:

- Handling changing priorities
- Work with tasks that volunteers choose themselves
- Keep its rules written down
- Protect review time
- Include quality and risk controls

Kanban meets the first four needs. It protects review time by limiting work in progress (WIP), the number of items allowed in a stage at once (Kanban University, n.d.). This assignment proposes placing that limit on review. Once it is reached, no new PRs enter review until waiting ones are cleared. Kanban also keeps its rules written down, because the conditions for moving work between stages are explicit and do not depend on who happens to be on the team. It also handles changing priorities and self-chosen tasks, because contributors still pull work from the voted backlog whenever they have time, and nobody has to commit to a fixed amount of work. Finally, Kanban is applied on top of an existing process without replacing it, so Actual keeps the workflow that already suits its volunteers.

However, Kanban doesn't meet the fifth need. It doesn't say how quality should be checked, and this matters because Actual's risk tolerance is low. The assignment therefore adds mandatory QA gates (defined in Section 3).

For more detail information, appendix A sets out how this choice was reached. Actual was profiled against eight standard selection factors, and Kanban was compared with Spiral, Scrum, Extreme Programming (XP) and DevOps. To briefly summarise it, Scrum and XP require team members to commit their time together, which volunteers cannot do. Spiral would place expert risk analysis on the already overloaded reviewers. And DevOps mainly adds automated testing, which Actual's CI already provides.

2.3 Applying Kanban to Actual

Figure 3 shows the proposed board. Each piece of work, such as a bug fix or a new feature, is a card that moves through six columns in order, from triage to operation. Each column has an exit policy, which is a written condition that must be met before work can be pulled into the next column (Ahmad et al., 2018, p. 98).

The WIP limit proposed in Section 2.2 sits on the Verify column, where review happens. It should be set from the number of reviews the maintainers actually complete, so that new work stops entering once reviews start to pile up.

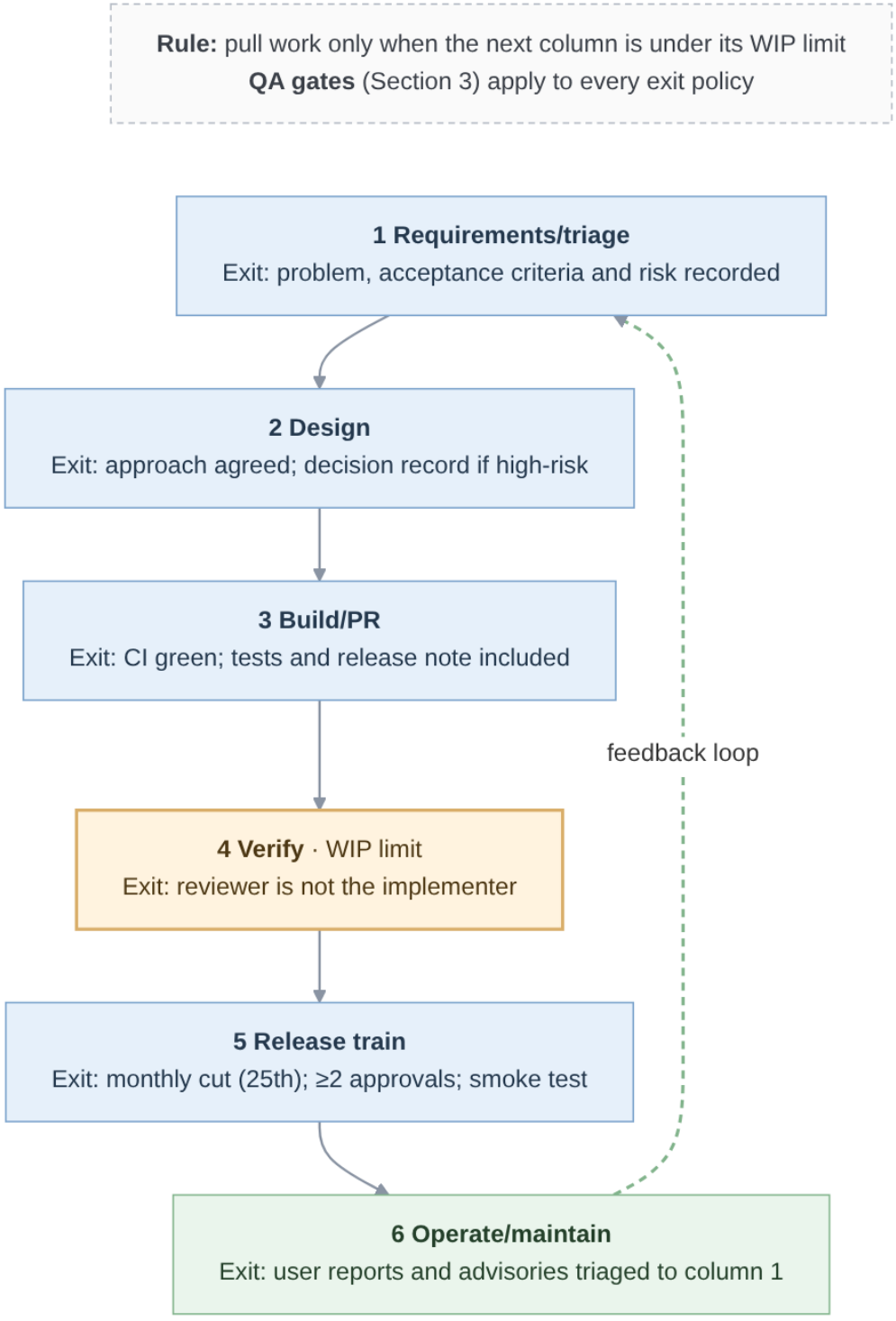


Figure 3. Proposed Kanban workflow for Actual with explicit exit policies.

2.4 Critical Evaluation

Kanban manages flow but does not improve quality by itself. It is "a relatively basic flow tool that needs to be supported by additional practices" (Ahmad et al., 2018, p. 110), which is why

the QA gates in section 3 are mandatory. Its WIP limit is also weaker in a volunteer project. It holds new PRs back and those PRs would wait like unanswered ones, which maintainers report discourages new contributors (Salkever, 2023, p. 5). However, PRs has already been waited months for review before any limit existed (Aboltins, 2025), so the limit makes the wait visible rather than creating it. Maintainers can also raise the limit at any moment if it starts to deter newcomers. The choice could also be revisited if Actual gains staff who can commit fixed hours in the future, which would make Scrum's commitments realistic, or runs its own hosted service which would give DevOps operations a chance to be implemented.

3.0 Quality Assurance Process

This section defines what quality means for Actual and which attributes matter the most, finds the gaps in the current practice, selects standards to close them, then builds the resulting plan into the Kanban workflow and examines its weaknesses.

3.1 What Quality Means for Actual

Quality can be defined from five viewpoints, adapted from Garvin (1984). And each of them describes what quality means for Actual:

- **User-based**
  For users, quality means balance and envelopes totals that are correct, private and easy to understand.
- **Product-based**
  Quality lies in properties of the software itself, such as access control on the sync server and the integrity of budget data.
- **Process-based**
  For Actual, quality means every change passes the same written checks, no matter who reviews it.
- **Value-based**
  Actual is free and funded by donations, so the quality work must be affordable in volunteer time.
- **Transcendent**
  In terms of transcendent view, quality refers to the sense that the app can be trusted with one's money. Users recognise it through daily use, and it will also be built up when the other four views above are met.

Together, these views give Actual's QA process one aim, which is to keep users' financial data correct and private, through checks that apply to every change but cost little volunteer time.

3.2 Key Quality Attributes

ISO/IEC 25010:2023 defines nine product quality characteristics (International Organization for Standardization & International Electrotechnical Commission, 2023). This report focuses on three of them: Security, Reliability and Interaction capability. Each was chosen because it meets two criteria, a failure would seriously harm users, by risking their money or data or locking them out of the app, and public evidence shows that Actual's current checks leave a gap (Section 3.3).

- **Security**
  Actual stores' user's transactions, balances and bank connections, which are some of the most private data a person has. If that data leaks or is changed without any permission, the users could face fraud or lose control of their finances.

- **Reliability**
  Users decide what to spend based on the balances Actual shows, so budget data must stay correct and in sync across devices, including after the app recovers from a failure. If it fails, users may stop trusting their numbers or be locked out of their own budget.
- **Interaction capability**
  Actual is used by normal people to manage their own money, so it must be used by people with different abilities and help them understand envelope budgeting. Actual already targets WCAG 2.1 AA (Actual Budget, 2026a), so accessibility is already a stated requirement.

3.3 Current QA Practices and Gaps

Actual already runs substantial automated QA. Every pull request triggers linting, type checking, Vitest unit tests and a database migration check, Playwright end-to-end and visual regression tests run on pull requests or in the merge queue (Actual Budget, 2026a, n.d.-i).

It also uses a semantic code analysis engine (CodeQL) to scan every pull requests and pushes (Actual Budget, 2026a). Moreover, releases will need approval from at least two other maintainers and do smoke tests (Actual Budget, n.d.-b). Actual's own security policy also asks for private vulnerability reporting and coordinated disclosure (Actual Budget, n.d.-g).

There are also four gaps that are discovered or happened from the research:

- **Security**
  No proactive check of access control before release. Three access-control flaws in the sync server feature were published in 2026 (Actual Budget, 2026b, 2026c, 2026d). The most critical one was about missing authentication on six of the SimpleFIN and Pluggy AI bank-sync endpoints, which the GoCardless endpoints had it (Actual Budget, 2026c). This shows inconsistent authentication across bank-sync providers, and the flaw was only found after release. The security policy also states no response time for reports (Actual Budget, n.d.-g).
- **Reliability**
  There is no test coverage measurement. Test coverage would not prove that the code is correct, but it would at least show what is untested. And reliability defects still reach users, like a sync defect that stopped some budgets from opening, and it's stayed in released versions for years (Brown, 2026). Although Actual's testing checklist asks for "appropriate test coverage" (Actual Budget, n.d.-i, Testing Checklist section), there's no coverage report found in the repository, and sync-server tests have coverage switched off (Actual Budget, 2026a). Therefore, nobody can tell how much the sync and migration code is actually tested and safe.
- **Interaction capability**
  No automated accessibility testing was found, despite the WCAG target (Actual Budget, 2026a).
- **Process**
  No written QA plan. Quality rules are spread across documentation, templates and CI files, so there's not a single place that states what must be checked before work moves on.

3.4 Selected Standards and Practices

Table 3.1 lists the standard that closes each gap from Section 3.3. Two standard are published standards that a volunteer team can use for free, while the rest are practices proposed in this report.

Gap	Standard or practice	How it is used
Security	OWASP ASVS 5.0.0	The ASVS chapters on authentication, sessions and authorisation cover the same kinds of flaw that was found in the 2026 advisories (OWASP Foundation, 2025).
Security	Authorisation test matrix (proposed)	An automated test goes through every server endpoint and checks that each one rejects requests that lack a valid login or the right permission.
Reliability	Coverage report	Coverage reports are switched on in the CI, so the team can see which parts of the sync and migration code have no tests.
Interaction capability	WCAG 2.2 AA	Actual's accessibility target moves from WCAG 2.1 AA to 2.2 AA, the version that W3C recommends for new accessibility work (World Wide Web Consortium, 2024).
Process	Written QA plan	Section 3.5 will sets out this plan.

Table 3.1. How each gap is closed.

3.5 Applying the QA Process within Kanban

This section turns the gaps found in Section 3.3 into the QA gates of Section 2.2, by adding them to the exit policies of the Kanban board (Section 2.3). As a result, a card can only move to the next column once its quality checks have passed.

When a change only touches wording or documents, running security and data checks on it would waste the review time that Section 2.1 already found scarce. Each card or task is therefore given a risk tier at triage stage, where it decides which QA checks the card must pass before it can move along the board. The tier is based on the most serious failure the change could cause:

- **High**
  A failure that could expose financial data or credentials, give wrong money figures or lose budget data. Such as authentication & authorisation, bank integration, budget and balance calculations or sync and encryption).
- **Standard**
  A failure would be visible but recoverable without data loss. Most of the features, such as reports, rules and UI components.
- **Low**
  A failure would be cosmetic like themes, wording and documentation.

These tiers were created for this report. They borrow the one idea from IEEE 1012-2024 standard: the more serious a component's possible failure, the more verification it needs (Institute of Electrical and Electronics Engineers [IEEE], 2025).

Table 3.2 shows the resulting QA plan, it explains the checks that are added to each Kanban column's exit policy, which cards they apply to, and who is responsible for what.

Kanban column	Checks added to the exit policy	Applies to	Who
Requirements/ triage	The card is given a risk tier, and acceptance criteria are written for security, data and accessibility wherever the change affects them.	All cards	Triager
Design	The endpoints and permissions the change touches are listed, and its effect on migrations and sync is stated.	High tier	Implementer
Build/PR	A coverage report is produced so untested code is visible, and any new endpoint is added to the authorisation test matrix.	All, except the matrix is for High tier	Implementer
Verify	Automated WCAG 2.2 AA checks must pass, and the selected ASVS requirements are checked against the change.	Cards that change the UI. And ASVS for High-tier security changes	Reviewer
Release train	A release cannot go out while any critical or high security finding is still open.	Every release	Release maintainers
Operate/ maintain	Security reports are answered. Each defect that reaches users gets a regression test, and the check that missed it is being reviewed.	Every report	Maintainers handling reports

Table 3.2. QA checks added to each Kanban column.

Figure 4 shows these checks on the board, together with the aim and quality objectives they serve.

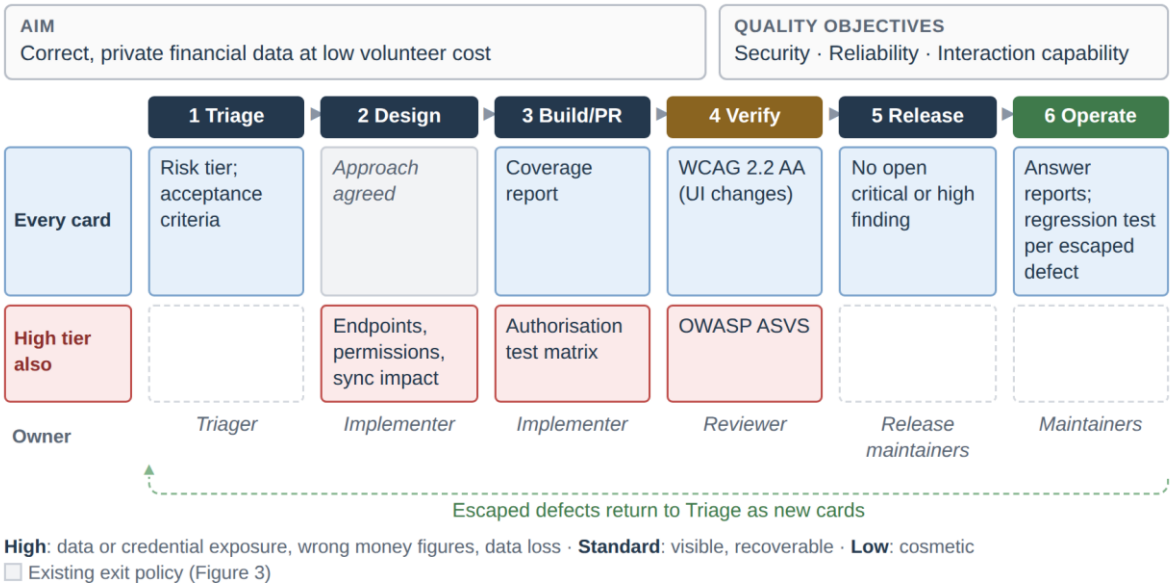


Figure 4. QA checks on the Kanban board, by risk tier.

As an example of a card moving through the plan in Table 3.2, consider a contributor who adds a new bank-sync provider, the kind of change behind GHSA-m2cq-xjgm-f668 (Section 3.3). At triage, the card is marked High tier because it touches bank sync, with an acceptance criterion that every new endpoint must refuse requests without a valid login. In Design, the implementer lists the new endpoints and who may call each one. At Build/PR, those endpoints are added to the authorisation test matrix. Without the login check, that test fails and the card cannot move to Verify, so a flaw like the 2026 one could be stopped before release. By contrast, a documentation typo fix is Low tier and passes through the normal checks only.

3.6 Critical Evaluation

The plan's main cost is volunteer time. Each check added to an exit policy is more work for a small team that already has little time for review (Section 2.1). The risk tiers keep this cost under control, since only High-tier cards go through the full set of checks. However, the tiers depend on judgement at triage, and a card given too low a tier will skip checks it needed. This matters most for security and data changes, where a missed check could reach users unnoticed.

4.0 Integrating IV&V with Kanban

This section examines how independently Actual's work is checked today, proposes an IV&V model built into the Kanban board from Section 2 and the QA plan from Section 3, and shows how the model meets the assignment's three goals, which are early defect detection, requirements compliance and overall quality.

4.1 Current V&V in Actual and Its Independence

IEEE 1012-2024 describes verification as checking whether the products of an activity conform to that activity's requirements, and validation as checking whether the product satisfies its intended use and user needs (IEEE, 2025). In independent V&V (IV&V), the work of each phase is checked by someone other than the person who built it.

Table 4.1 applies this test to each of Actual's existing V&V activities, from requirements through to maintenance. *Partial* means independent but limited, because it is covered by automated tests, and nobody has to test the preview builds too.

Activity	Phase	Built by	Checked by	Checker separate from builder?
Requirements and design check	Requirements, design	Contributor proposing the change	None found in the documentation (October 2026)	—
PR review	Implementation	Implementer	Another core maintainer	Yes
CI tests and CodeQL scanning	Implementation, testing	Implementer	Automated checks configured by the team (Actual Budget, 2026a)	Partial
PR preview and nightly builds	Testing	Implementer	Any community member, voluntarily (Actual Budget, n.d.-f)	Partial
Release approval and smoke test	Deployment	Maintainer cutting the release	At least two other maintainers approve and follow up with smoke tests (Actual Budget, n.d.-b)	Yes
Vulnerability reports	Maintenance	Core team	Reporters outside the core team (Actual Budget, 2026b, 2026c, 2026d)	Yes, but only after release
External audit	Maintenance	—	No public evidence found in the documentation and blog (October 2026)	—

Table 4.1. Independence of Actual's existing V&V activities.

Actual already has some independent checking, but it is ad hoc. Nobody other than the contributor checks requirements or design, and how closely a change is checked depends on who happens to review it, whatever its risk is. Some of the checking also comes late after release, such as the three 2026 advisories in Section 3.3 which were found in released versions, by reporters outside the core team (Actual Budget, 2026b, 2026c, 2026d).

4.2 IV&V Integration Model for Kanban

The proposed IV&V model makes independent checking depend on a change's risk instead of on whoever happens to review it. Some of these checks happen inside the team and some come from outside it.

Within the team, High-tier cards (Section 3.5) get an independent reviewer. This is a maintainer who did not write the change and does not own that area in Actual's CODEOWNERS file (Actual Budget, 2026a). Other cards keep the usual review, which protects the few reviewers that Section 2 found to be the bottleneck.

However, the team's own review can still miss things, as the three 2026 advisories that were in the released code and were found by people outside of the core team (Section 3.3). The project's own private vulnerability reporting policy already lets outsiders to report flaws like these, but it only happens once the code has already shipped. Therefore, the proposed model adds an occasional external security audit to the sync server and other high-risk area. For example, the Open Source Technology Improvement Fund (2026) arranges audits like this for open-source projects, which are paid for by outside funders, so it wouldn't use volunteer time or Actual's donations.

Table 4.2 shows where each independent check sits on the Kanban board. A card cannot move on until the condition in the exit policy is met.

Kanban column	Who checks	What they check	Exit policy
Requirements/ triage	A triager who didn't request the change	Checks if the acceptance criteria can be tested, and which risk tier the card needs (all cards)	The tier and testable criteria are recorded
Design	The independent reviewer	Checks the planned endpoints and permissions against the ASVS requirements that apply	The reviewer accepts the design
Build/PR	Automated CI checks, set up by the team	Runs the automated tests and scans on every PR, including the authorisation test matrix	All checks pass

Kanban column	Who checks	What they check	Exit policy
Verify	The independent reviewer, and someone other than the implementer for validation	Checks the code against the ASVS requirements, the authorisation test matrix and the acceptance criteria, and tries it on a preview build. For sync or migration changes, also checks that balances stay correct	The approval lists what was checked, and no critical or high finding is open
Release train	One of the approving maintainers	Checks if every High-tier card in the release passed Verify	The evidence is confirmed before the release version is tagged
Operate/ maintain	Outside reporters and the optional funded auditor	Check released code for flaws that the team missed. For each fixed flaw, a reviewer other than the code owner also checks whether its area was tiered too low	The advisory is published, a regression test is merged and the tier rule is corrected if needed

Table 4.2. Independent checks on the Kanban board (High-tier cards unless stated).

Standard-tier cards skip the design review and only need checking on a preview build, while Low-tier cards follow normal review.

4.3 How the Integration Meets the Three Goals

This section will shows how the proposed model would meets the three goals named at the start of Section 4 by using Actual's own evidence.

Early defect detection is most obvious one to see with the critical 2026 bank-sync flaw, which was only found after release (Section 3.3). Bank sync is High tier (Section 3.5), so under the model a change like this would have met the independent reviewer twice before release. One is at Design stage, they would have flagged an endpoint planned without a login check. While the second one at Verify stage, which they would have checked that every endpoint was in the authorisation test matrix, because an implementer who forgets a login check might also forget its test too.

Requirements compliance means showing that what users asked for was built and tested. In Actual, a feature request starts as an issue, the PR template asks the contributor to link that issue, and a release note describes what shipped (Actual Budget, 2026a). However, no step checks that the finished PR actually does what the issue asked for. Under the proposed model, someone other than the requester makes sure that at triage, each acceptance criterion can be tested, and the reviewer only approves at Verify once every criterion has a passing test.

Lastly, the overall quality will improves because the riskiest code is being checked independently before releases, while small changes still go through a single review. And any

defect that still gets through will becomes a regression test, and the tier of its area will be checked again.

References

1. Aboltins, M. J. (2025, January 2). *Roadmap for 2025*. Actual Budget. https://actualbudget.org/blog/roadmap-for-2025/

2. Actual Budget. (n.d.-a). *Contributing to Actual Budget*. Retrieved October 22, 2026, from https://actualbudget.org/docs/contributing/

3. Actual Budget. (n.d.-b). *How to cut a release*. Retrieved October 22, 2026, from https://actualbudget.org/docs/contributing/releasing/

4. Actual Budget. (n.d.-c). *Installing Actual*. Retrieved October 22, 2026, from https://actualbudget.org/docs/install/

5. Actual Budget. (n.d.-d). *New core contributors guide*. Retrieved October 22, 2026, from https://actualbudget.org/docs/contributing/leadership/new-core-contributors-guide/

6. Actual Budget. (n.d.-e). *Paying reviewers for administrative work*. Retrieved October 22, 2026, from https://actualbudget.org/docs/contributing/leadership/paying-contributors/

7. Actual Budget. (n.d.-f). *Preview builds*. Retrieved October 22, 2026, from https://actualbudget.org/docs/contributing/preview-builds/

8. Actual Budget. (n.d.-g). *Security policy* [SECURITY.md]. GitHub. Retrieved October 22, 2026, from https://github.com/actualbudget/.github/blob/main/SECURITY.md

9. Actual Budget. (n.d.-h). *Syncing across devices*. Retrieved October 22, 2026, from https://actualbudget.org/docs/getting-started/sync/

10. Actual Budget. (n.d.-i). *Testing*. Retrieved October 22, 2026, from https://actualbudget.org/docs/contributing/testing/

11. Actual Budget. (n.d.-j). *A tour of Actual*. Retrieved October 22, 2026, from https://actualbudget.org/docs/tour/

12. Actual Budget. (2026a). *Actual* [Computer software]. GitHub. https://github.com/actualbudget/actual

13. Actual Budget. (2026b, June 12). *Disabled OpenID users keep access through existing session tokens* (GHSA-cq9c-6w48-qmfg) [Security advisory]. GitHub. https://github.com/actualbudget/actual/security/advisories/GHSA-cq9c-6w48-qmfg

14. Actual Budget. (2026c, February 20). *Missing authentication for SimpleFIN and Pluggy AI bank sync endpoints* (GHSA-m2cq-xjgm-f668) [Security advisory]. GitHub. https://github.com/actualbudget/actual/security/advisories/GHSA-m2cq-xjgm-f668

15. Actual Budget. (2026d, April 23). *Privilege escalation via 'change-password' endpoint on OpenID-migrated servers* (GHSA-prp4-2f49-fcgp) [Security advisory]. GitHub. https://github.com/actualbudget/actual/security/advisories/GHSA-prp4-2f49-fcgp

16. Ahmad, M. O., Dennehy, D., Conboy, K., & Oivo, M. (2018). Kanban in software engineering: A systematic mapping study. *Journal of Systems and Software, 137*, 96–113. https://doi.org/10.1016/j.jss.2017.11.045

17. Beck, K., & Andres, C. (2005, June 10). *Getting started with eXtreme Programming: Toe dipping, racing dives, and cannonballs: Appendix: Primary practices*. InformIT. https://www.informit.com/articles/article.aspx?p=390816&seqNum=3

18. Boehm, B. W. (1988). A spiral model of software development and enhancement. *Computer, 21*(5), 61–72. https://doi.org/10.1109/2.59

19. Brown, S., II. (2026, September 15). *[AI] fix(sync): receive only the latest timestamp to stop counter overflow* (#8706) [Commit]. GitHub. https://github.com/actualbudget/actual/commit/5f60f2971ca0f4a84133ba43a047a9f7b029d409

20. DORA. (n.d.). *Test automation*. Retrieved October 26, 2026, from https://dora.dev/capabilities/test-automation/

21. Garvin, D. A. (1984). What does "product quality" really mean? *Sloan Management Review, 26*(1), 25–43. https://sloanreview.mit.edu/article/what-does-product-quality-really-mean/

22. GeeksforGeeks. (2025, July 23). *How to choose the best SDLC model for your project*. https://www.geeksforgeeks.org/software-engineering/how-to-choose-the-best-sdlc-model-for-your-project/

23. Institute of Electrical and Electronics Engineers. (2025). *IEEE standard for system, software, and hardware verification and validation* (IEEE Std 1012-2024). https://standards.ieee.org/ieee/1800.2/7324/

24. International Organization for Standardization & International Electrotechnical Commission. (2023). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model* (ISO/IEC Standard No. 25010:2023). https://www.iso.org/standard/78176.html

25. Kanban University. (n.d.). *The official guide to the Kanban method*. Retrieved October 22, 2026, from https://kanban.university/kanban-guide/

26. Open Source Technology Improvement Fund. (2026, January 30). *2025 annual report*. https://ostif.org/2025-annual-report/

27. OWASP Foundation. (2025). *OWASP application security verification standard* (Version 5.0.0). https://owasp.org/www-project-application-security-verification-standard/

28. Salkever, A. (2023). *Open source maintainers: Exploring the people, practices, and constraints facing the world's most critical open source software projects*. The Linux Foundation. https://doi.org/10.70828/lddj8886

29. Schwaber, K., & Sutherland, J. (2020). *The Scrum guide*. https://scrumguides.org/scrum-guide.html

30. Stacoviak, A., & Santo, J. (Hosts). (2022, July 1). *Actual(ly) opening up with James Long* (No. 495) [Audio podcast episode]. In *The Changelog*. Changelog Media. https://changelog.com/podcast/495

31. World Wide Web Consortium. (2024, December 12). *Web content accessibility guidelines (WCAG) 2.2* (W3C Recommendation). https://www.w3.org/TR/WCAG22/

32. youngcw. (2025, December 20). *Roadmap for 2026*. Actual Budget. https://actualbudget.org/blog/roadmap-for-2026/

Appendix A: Methodology Selection

Supporting detail for Section 2.2: how Kanban was selected.

A.1 Project Characteristics and Requirements

Selecting a software process model involves assessing project characteristics against eight standard factors (GeeksforGeeks, 2025). These factors indicate whether a project requires an iterative, agile model suited to changing requirements and direct user involvement, or a linear model such as Waterfall suited to predictable requirements, fixed deadlines, and stability. Table A.1 evaluates Actual against these criteria.

Factor	Actual	Points to
Project requirements	Evolving: new requests arrive continuously and are re-ranked by users' votes (Actual Budget, 2026a)	Agile
Size and complexity	Small: eight maintainers plus volunteers (Actual Budget, n.d.-a). Waterfall suits small projects only when requirements are straightforward, which Actual's are not	Agile
Flexibility and adaptability	Priorities shift with votes, and each monthly release ships whatever is ready (Actual Budget, n.d.-b)	Agile
Customer involvement	Users shape the work continuously through requests and votes (Actual Budget, 2026a)	Agile
Risk tolerance	Low: a wrong balance can lead to a poor spending decision, and a security flaw can expose financial data (Actual Budget, 2026c)	Linear
Time constraints	A fixed monthly release date, but no feature is promised for it (Actual Budget, n.d.-b)	Agile
Team expertise	Cross-functional volunteers who choose their own work (Actual Budget, n.d.-a, n.d.-d); the few experienced reviewers have been overloaded before (Aboltins, 2025)	Agile, but under strain
Regulatory compliance	No regulatory requirements found in Actual's documentation (October 2026)	No constraint

Table A.1. Actual against the selection factors.

While most factors support an agile approach, team expertise and risk tolerance complicate the choice. Agile methods rely on an experienced, self-disciplined team, whereas low risk tolerance favours linear models (GeeksforGeeks, 2025). Actual has only a few experienced volunteer reviewers, and their capacity has failed under load previously (Aboltins, 2025). Furthermore, a wrong balance can lead a household to a poor spending decision, and in 2026 a critical security flaw permitted unauthenticated access to users' bank balances and transactions (Actual Budget, 2026c).

Because standard guidance recommends combining approaches when project characteristics diverge (GeeksforGeeks, 2025), Actual requires an agile workflow supplemented with written controls where volunteer capacity and risk tolerance leave it exposed. Table A.2 maps these considerations to five specific requirements tested in Section A.3.

Requirement	Why Actual needs it	Brief's lens
Handles changing priorities	Votes keep re-ranking the work (project requirements, flexibility)	Stakeholder expectations
Works with self-chosen tasks	Volunteers cannot be assigned work (team expertise)	Constraints
Keeps its rules written down	Actual's core team has changed over time (Aboltins, 2025), so practices held only in people's heads would be lost (team expertise)	Risks
Protects review time	Reviewers are the bottleneck, and maintainers link administrative overload to burnout (Salkever, 2023, pp. 26–27) (team expertise)	Constraints, risks
Includes quality and risk controls	Defects affect money and private data, so low risk tolerance calls for more structure (risk tolerance)	Goals, risks

Table A.2. Requirements for Actual's methodology.

A.2 Candidate Methodologies

Five methodologies reflect distinct approaches to software delivery: Spiral, Scrum, Extreme Programming (XP), Kanban, and DevOps. Linear models such as Waterfall and the V-Model were excluded because they suit stable requirements and offer little flexibility (GeeksforGeeks, 2025), which conflicts with Actual's vote-driven backlog. Rapid Application Development (RAD) and the Rational Unified Process (RUP) were also set aside because their structured phases and scheduled deliverables require assignable staff rather than self-directed volunteers. Although Spiral is not an agile methodology, it is evaluated because Actual's low risk tolerance favours systematic risk analysis.

Actual's existing workflow serves as a baseline for the evaluation. Any adopted methodology must demonstrate clear improvements over current practice.

A.3 Evaluation and Selection

Each candidate is assessed against the requirements in Table A.2, assuming that existing practices (specifically continuous integration, peer review, and monthly releases) remain in place. The assessment distinguishes between two outcomes:

- a clash occurs when a candidate requires conditions that Actual cannot provide, disqualifying the methodology;
- a gap occurs when a candidate omits a required practice, which can be resolved by introducing supplementary controls.

Table A.3 outlines the demands, clashes, and gaps identified for each candidate.

Candidate	What it requires of the team	Clash with how Actual works	Requirement left uncovered (gap)
Spiral	Repeated cycles, each starting with expert analysis of the largest remaining risk	The analysis would fall on the overloaded reviewers	Handles changing priorities
Scrum	Sprints of up to one month, a Sprint Goal the team commits to, a single Product Owner who orders the work, a daily 15-minute meeting	Needs a committed Sprint Goal and a single Product Owner	Protects review time
XP	Weekly planning, test-first development, pair programming, a team that sits together	Needs developers working side by side at the same time	Includes quality and risk controls (security, accessibility)
Kanban	A visible board, work-in-progress (WIP) limits and written rules for moving work, applied on top of an existing process	None found	Includes quality and risk controls
DevOps	Shared responsibility and automation across building and running the software	None found	Protects review time
Current process (reference)	Already in place	None	Keeps its rules written down (partly: the route is documented, but not the conditions for moving work on); protects review time; includes quality and risk controls

Table A.3. What each candidate requires, and its clashes and gaps.

Three candidate methodologies clash directly with Actual's volunteer structure. Scrum and XP require team members who can coordinate schedules and commit time collectively. In Scrum, the Sprint Goal "is a commitment by the Developers" (Schwaber & Sutherland, 2020, Commitment: Sprint Goal section) and the Product Owner "is one person, not a committee" (Schwaber & Sutherland, 2020, Product Owner section). Similarly, XP relies on pair programming, which requires two developers to work simultaneously even when collaborating remotely (Beck & Andres, 2005). Because Actual's contributors volunteer irregularly without schedule commitments, and priorities are determined by community voting rather than an individual owner, these core assumptions fail.

Spiral addresses low risk tolerance by commencing each cycle with risk analysis. However, effective risk evaluation demands experienced technical judgement (Boehm, 1988), which is why Spiral is usually described as complex and dependent on specialist expertise

(GeeksforGeeks, 2025). In Actual's case, that analytical burden would fall on the small group of maintainers who already represent the project's primary bottleneck.

Neither Kanban nor DevOps clashes with Actual's working model, shifting the evaluation to what each adds beyond existing practice. DevOps emphasises continuous automated testing across delivery (DORA, n.d.), which Actual already maintains within its CI pipeline, alongside closer operational integration. Operational integration offers limited benefit here, as end users host the server independently or through third-party hosting services (Actual Budget, n.d.-c). Consequently, DevOps leaves maintainer review capacity unaddressed.

Kanban directly addresses the bottleneck identified in Section 2.1, where nothing limits how many pull requests wait for review. By capping work in progress (WIP), Kanban restricts the number of active items permitted in any workflow stage (Kanban University, n.d.). Applying this limit to the review column ensures that once the threshold is reached, new submissions cannot enter review until pending requests are processed, thereby protecting maintainer time. Furthermore, establishing explicit, written policies for transitioning work between stages preserves process knowledge despite changes in team membership. Contributors can continue selecting issues from the voted backlog whenever their availability permits, preserving volunteer flexibility without requiring fixed sprint commitments.

Compared to maintaining the status quo, Kanban introduces structured WIP limits and explicit transition rules while preserving the autonomy volunteer contributors require. However, core Kanban does not define technical quality verification or prescribe required test standards. Because Actual operates with low risk tolerance regarding financial accuracy and privacy, process oversight must be strengthened. This report therefore incorporates mandatory quality gates (QA gates), embedding the risk-scaled verification criteria detailed in Section 3 into each column's exit policy. The selected methodology is Kanban, tailored with mandatory QA gates.

Appendix B: SQAP Summary Poster

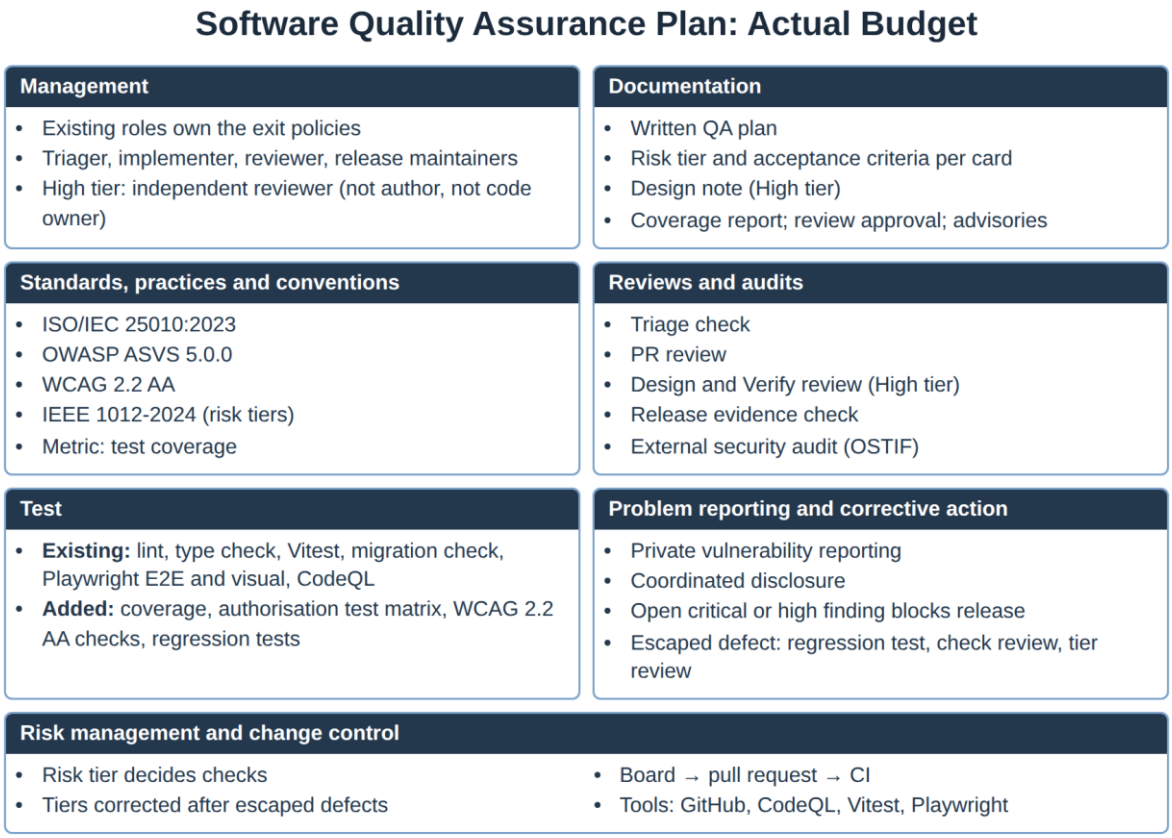


Figure B.1. Software Quality Assurance Plan for Actual Budget.